

Relatório Projeto #1 AED 2025-2026

Nome: Tomás Simões Correia
PL (inscrição): 1

Nº Estudante: 2024198175

- ⇒ Registrar os tempos computacionais das 3 soluções desenvolvidas.
- ⇒ Os tamanhos dos arrays (N) devem permitir obter dados representativos (podem ser, por exemplo: 100000, 200000, 500000, 750000, 1000000).
- ⇒ A análise deve ser realizada para dois tipos de sequências:
 - o Sequências onde existe pelo menos um par de valores cuja soma é igual a k;
 - o Sequências onde não existe qualquer par cuja soma seja k.
- ⇒ Só deve ser contabilizado o tempo do algoritmo (exclui-se o tempo de leitura/geração do input e de impressão dos resultados).
- ⇒ Sugere-se a realização de várias medições (pelo menos 5) para cada solução e apresentação da média.
- ⇒ Devem apresentar e discutir as regressões para as 3 soluções, incluindo também o coeficiente de determinação/regressão (R^2).

Tabela para as 3 soluções

Tabela para o pior caso (não foi encontrado um par de valores cuja soma é igual a k), k = -1

Sequências (nº de elementos)		Solução exaustiva (tempo de execução, s)	Solução com ordenamento (tempo de execução, s)	Solução elaborada (tempo de execução, s)
10 000	0	3.145833730697632	0.0019404888153076172	0.001070261001586914
	1	3.107760190963745	0.0019197463989257812	0.0009481906890869141
	2	2.941826343536377	0.0019145011901855469	0.0007920265197753906
	3	3.1374919414520264	0.00197601318359375	0.0007998943328857422
	4	3.1533493995666504	0.0019683837890625	0.0007739067077636719
	Média	3.09725232124329	0.00194382667542	0.00087685585022
32 500	0	31.8753132593917847	0.007066011428833008	0.005079507827758789
	1	33.59806561470032	0.006929636001586914	0.004045963287353516
	2	33.995646476745605	0.007009744644165039	0.003149271011352539

	3	33.71545720100403	0.0069997310638427734	0.004199504852294922
	4	32.742334604263306	0.006980419158935547	0.003114938735961914
	Média	33.1853634312210	0.0069971084595	0.0039178371429
55 000	0	100.84361743927002	0.012150049209594727	0.007364749908447266
	1	99.99515128135681	0.012104511260986328	0.006470441818237305
	2	99.39670467376709	0.012049436569213867	0.006180286407470703
	3	97.9942774772644	0.012078523635864258	0.006340980529785156
	4	95.88275051116943	0.012103796005249023	0.006177186965942383
	Média	98.8225002765656	0.0120972633362	0.0065067291260
75 500	0	219.95104098320007	0.018385887145996094	0.012932777404785156
	1	237.38093066215515	0.017894744873046875	0.014168024063110352
	2	219.83259677886963	0.017604589462280273	0.013058900833129883
	3	202.03246569633484	0.017560243606567383	0.011044502258300781
	4	251.70056557655334	0.01973438262939453	0.014532089233398438
	Média	226.1795199394230	0.0182359695435	0.0131472587585
100 000	0	425.61484837532043	0.024251222610473633	0.02030038833618164
	1	473.2432119846344	0.024674654006958008	0.019290447235107422
	2	464.51516556739807	0.023772001266479492	0.016269683837890625
	3	449.50502490997314	0.02356553077697754	0.015949487686157227
	4	461.95178627967834	0.02469611167907715	0.017443180084228516
	Média	454.9660074234010	0.0241919040680	0.0178506374359

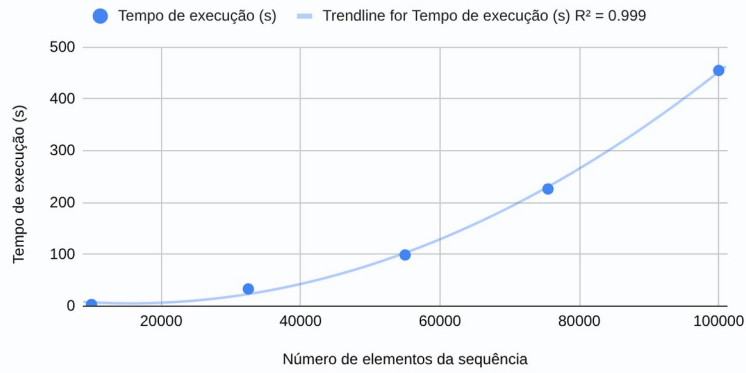
Tabela para o caso médio (foi encontrado um par de valores cuja soma é igual a k)

Sequências (nº de elementos)		Solução exaustiva (tempo de execução, s)	Solução com ordenamento (tempo de execução, s)	Solução elaborada (tempo de execução, s)
10 000	0, k = 3107	0.0006456375122070312	0.0016436576843261719	1.8358230590820312e-05
	1, k = 8478	0.00017571449279785156	0.0011408329010009766	2.7418136596679688e-05
	2, k = 3529	0.00019931793212890625	0.0016231536865234375	2.7179718017578125e-05
	3, k = 9519	0.00011658668518066406	0.0010116100311279297	9.059906005859375e-06
	4, k = 6130	0.0004506111145019531	0.0013506412506103516	4.887580871582031e-05
	Média	0.0003175735474	0.0013539791107	0.0000261920000
32 500	0, k = 26906	0.0002536773681640625	0.00441288948059082	3.933906555175781e-05

	1, k = 25773	0.0005910396575927734	0.004475831985473633	2.193450927734375e-05
	2, k = 27093	0.0005066394805908203	0.00452423095703125	3.886222839355469e-05
	3, k = 14823	0.0009102821350097656	0.005592823028564453	3.337860107421875e-05
	4, k = 8941	0.0001633167266845703	0.006199836730957031	2.8848648071289062e-05
	Média	0.0004849910736	0.0050411224365	0.0000324600000
55 000	0, k = 51218	0.0049664974212646484	0.007201433181762695	7.534027099609375e-05
	1, k = 722	0.05505800247192383	0.01240682601928711	0.00021529197692871094
	2, k = 43035	0.0002818107604980469	0.007938146591186523	1.049041748046875e-05
	3, k = 50666	0.0005919933319091797	0.0071828365325927734	5.459785461425781e-05
	4, k = 6352	0.005525827407836914	0.011698246002197266	0.00015807151794433594
	Média	0.0132848262787	0.0092854976654	0.0001027526990
75 500	0, k = 12022	0.01570749282836914	0.0172421932220459	0.000179290771484375
	1, k = 59196	0.00033020973205566406	0.012524843215942383	4.887580871582031e-05
	2, k = 53003	0.0033121109008789062	0.012506961822509766	5.936622619628906e-05
	3, k = 77430	0.0008759498596191406	0.010003805160522461	0.00010824203491210938
	4, k = 40460	0.0014886856079101562	0.014668703079223633	7.581710815429688e-05
	Média	0.0043428897858	0.0133893013000	0.0000943265613
100 000	0, k = 6954	0.04916214942932129	0.0247800350189209	0.0001437664031982422
	1, k = 5246	0.0263211727142334	0.02517557144165039	7.915496826171875e-05
	2, k = 85499	0.0008327960968017578	0.015619993209838867	7.748603820800781e-05
	3, k = 36492	0.0115966796875	0.020840883255004883	0.00011777877807617188
	4, k = 47819	0.0009813308715820312	0.020725488662719727	6.389617919921875e-05
	Média	0.0177788257599	0.0214283943176	0.0000964290363

Gráfico para a solução A

Tempo de execução da solução exaustiva (par não encontrado)



Tempo de execução da solução exaustiva (par encontrado)

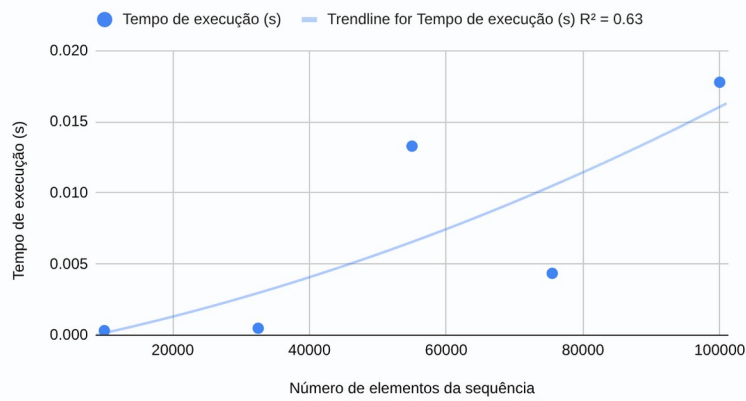
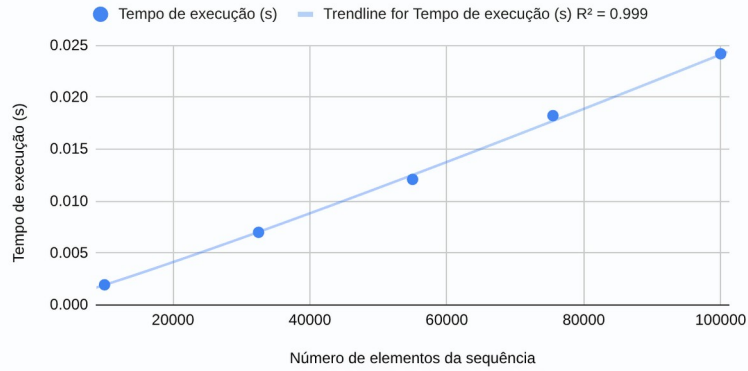


Gráfico para a solução B

Tempo de execução da solução com ordenamento (par não encontrado)



Tempo de execução da solução com ordenamento (par encontrado)

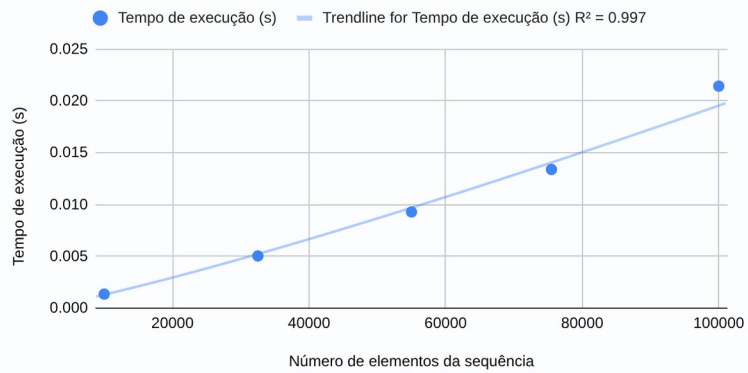
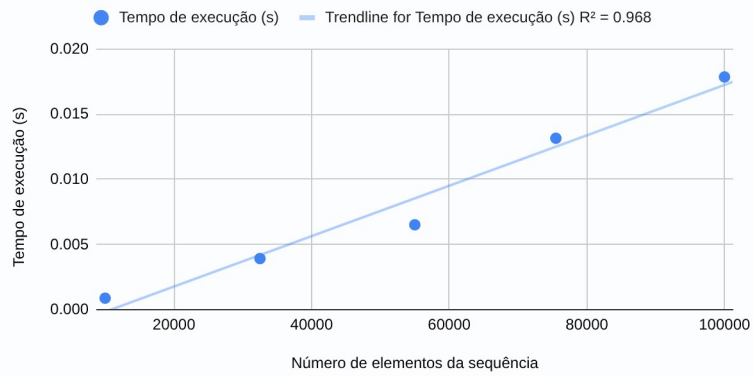
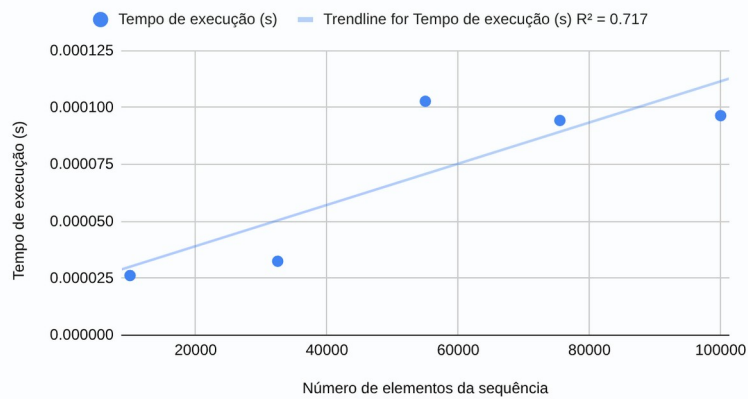


Gráfico para a solução C

Tempo de execução da solução elaborada (par não encontrado)



Tempo de execução da solução elaborada (par encontrado)



Análise dos resultados tendo em conta as regressões obtidas e como estas se comparam com as complexidades teóricas:

Nota: As seguintes afirmações têm como prova os gráficos das soluções no pior caso possível: quando nenhum par é encontrado cuja soma resulte em k (será explicado a razão a seguir). As complexidades teóricas das 3 soluções são as seguintes:

- **Solução exaustiva:** $O(n^2)$, pois esta solução envolve percorrer a sequência 2 vezes, aumentando quadraticamente o tempo de execução.
- **Solução com ordenamento:** $O(n \log(n))$, pois a parte de ordenar a sequência foi feita através do método `sort()` do Python, que tem uma complexidade de tempo de $O(n \log(n))$, enquanto o código seguinte de procura do par percorre no máximo a sequência 1 vez (ou seja, $O(n)$), sendo então a complexidade de tempo prevalente o do método `sort()`.
- **Solução elaborada:** $O(n)$, pois percorre a sequência apenas 1 vez e a procura de elementos na estrutura de dados “set” do Python tem uma complexidade de tempo de $O(1)$.

Como também se pode reparar, os pontos dos gráficos do pior caso possível estão mais próximos da reta de regressão (têm um R^2 mais próximo de 1, ou seja, têm resultados mais próximos do esperado, sendo por isso o alvo de comparação no parágrafo anterior) que os dos gráficos em que um par cuja soma seja igual a k é encontrado. Isto deve-se ao facto que quando estes algoritmos encontram um par que valide a sequência, eles terminam de a ler. Isto pode acontecer em qualquer altura da sequência, podendo terminar cedo, tendo um tempo de execução mais baixo ou mais tarde, tendo um tempo de execução mais elevado. Por isso, para ter resultado mais fiáveis, teria de se ter testado muito mais que 5 vezes cada tamanho de sequência e testar opcionalmente com ainda mais tamanhos de sequência diferentes.

Código:

```
```python
from typing import Callable

from time import time

from os import path

import random

import csv

def exhaustiveSolution(input: list[int], k: int) -> bool:

 for i in input:
 for j in input:
 if i + j == k:
 return True

 return False

def orderedSolution(input: list[int], k: int) -> bool:

 input.sort()

 endIndex: int = len(input) - 1
 startIndex: int = 0

 for _ in range(len(input) - 1):
 if startIndex >= endIndex:
 return False

 result: int = input[startIndex] + input[endIndex]

 if result == k:
 return True
 elif result > k:
 endIndex -= 1
 continue

 startIndex += 1

 return False
```



```
def elaborateSolution(input: list[int], k: int) -> bool:
```

```
 inputSet: set[int] = set()
```

```
 for num in input:
```

```
 if k - num in inputSet:
```

```
 return True
```

```
 inputSet.add(num)
```

```
 return False
```

```
def testAlgorithms(
```

```
 algorithms: dict[str, Callable[[list[int], int], bool]],
```

```
 i: int,
```

```
 inputSize: int,
```

```
 input: list[int],
```

```
 isValid: bool,
```

```
 basePath: str,
```

```
) -> None:
```

```
 k: int = 0
```

```
 if isValid:
```

```
 # Existe exatamente 1 '0' em cada input, por isso se k for um
```

```
 # dos elementos do input, irá ser calculado $0 + k = k$, tornando
```

```
 # o input válido
```

```
 k = random.randint(0, inputSize)
```

```
 else:
```

```
 # Não existem números negativos nos inputs, logo com este k, irá
```

```
 # ser retornado sempre "False"
```

```
 k = -1
```

```
 for fileNamePrefix, algorithm in algorithms.items():
```

```
 fileName: str = f"{fileNamePrefix}-{inputSize}-{i}-"
```

```
 if isValid:
```

```
 fileName += "Avg.txt"
```

```
 else:
```

```
 fileName += "Worst.txt"
```

```
 with open(path.join(basePath, "results", fileName), "w") as algorithmFile:
```

```
 algorithmFile.write(f"k = {k}\n")
```

```

print(
 f"Começando o algoritmo {fileNamePrefix} com um input de {inputSize} elementos e k ",
 end="",
)
if isValid:
 print("válido")
else:
 print("inválido")

inputCopy: list[int] = input.copy()
start: float = time()
inputIsCorrect: bool = algorithm(inputCopy, k)
end: float = time()
elapsedTimeSeconds: float = end - start

algorithmFile.write(f"Resultado: {inputIsCorrect}\nTempo: {elapsedTimeSeconds} s\n")

def main() -> None:
 basePath: str = path.dirname(__file__) # Diretório onde está este script
 algorithms: dict[str, Callable[[list[int], int], bool]] = {
 "exhaustive": exhaustiveSolution,
 "ordered": orderedSolution,
 "ellaborate": ellaborateSolution,
 }

 inputsDict: dict[int, list[list[int]]] = {
 10_000: [],
 32_500: [],
 55_000: [],
 77_500: [],
 100_000: [],
 }

 for inputSize, inputs in inputsDict.items():
 for i in range(5):
 input: list[int] = random.sample(range(inputSize), inputSize)
 inputs.append(input)

```

```
inputsFileName = f"{inputSize}_{i}.csv"

with open(path.join(basePath, "inputs", inputsFileName), "w") as file:
 csv.writer(file).writerow(input)

for i, input in enumerate(inputs):
 for isValid in (False, True):
 testAlgorithms(algorithms, i, inputSize, input, isValid, basePath)

print("Terminou")

if __name__ == "__main__":
 main()
...
```